



DIGILOG 360

Platform Wiki & Operations Manual

The complete guide to the DigiLog 360 multi-tenant security operations platform — architecture, technology stack, every role, every workflow, and step-by-step instructions for every task on web and mobile.

Admin Console · Next.js 15

Mobile App · Expo / React Native

Backend · Supabase

6 Roles

Multi-Tenant

Realtime SLA

Offline-First Mobile

Email Alerts · Resend

Netstream Intergrated Solutions

Proprietary & Confidential — © 2026 Kruz Naidoo. All Rights Reserved.

Edition 1.1

August 2026

BEFORE YOU BEGIN

About This Manual

What this document covers, who each part is written for, and the conventions used throughout.



DigiLog 360 replaces the paper occurrence book, the patrol clock, the visitor register, and the key register with one connected platform. This manual is both a **training guide** for every role and a **reference** for administrators and operators of the platform.

Who should read what

YOU ARE...	READ FIRST	THEN
Guard	Ch. 6 (Signing in) · Ch. 8 (Guard playbook)	Ch. 15, 17, 18, 19 for feature detail
Supervisor	Ch. 8 + Ch. 9 (Supervisor playbook)	Ch. 14, 16, 17
Control Room	Ch. 10 (Control Room playbook)	Ch. 14–16, 21, 22
Manager	Ch. 11 (Manager playbook)	Ch. 14, 21
Administrator	Ch. 12 (Admin playbook)	Ch. 5, 16, 23
Super User	Ch. 13 (Super User playbook)	Everything — you run the platform
Engineer / IT operator	Ch. 2–5	Ch. 23–25 + PRODUCTION.md in the repo

TERMINOLOGY

An **occurrence** is any logged incident or event — the digital equivalent of an entry in the paper Occurrence Book. Each one receives a sequential **OB number** (e.g. `OB-2026-04180`) the moment it is saved. "Occurrence" and "incident" are used interchangeably in this manual.

Conventions

Navigation paths

Menu locations are written as [Sidebar](#) → [Occurrences](#) → [Log New Occurrence](#) for the web console and [Home](#) → [Quick actions](#) → [Log occurrence](#) for mobile.

Capability keys

Every gated feature lists its capability key, e.g. `occurrences.log`. If you can't see a screen described here, that capability isn't granted to your role — see Chapter 5.

Badges

Severity and status appear exactly as in the apps:



Callouts

Tip = a faster way to work · **Note** = background you should know · **Warning** = an action that is hard to undo.

Related documents in the repository

README.md

The engineering reference: full schema, migration timeline, RLS helpers, edge-function internals and demo-account details.

PRODUCTION.md

The operator runbook: deployment, secret rotation cadence, monitoring setup, backup / disaster recovery and the pre-release checklist.

PERFORMANCE_OPTIMIZATIONS.md

Measured before/after numbers behind the console's speed: bundle diet, parallel fetching, realtime instead of polling, composite indexes.

KEEPING THIS MANUAL CURRENT

This wiki is generated from [docs/wiki/DigiLog360-Wiki.html](#) in the repository. When a feature lands, edit that file and re-print it to PDF — the source of truth stays in version control alongside the code it documents.

CONTENTS

Table of Contents



Five parts: the platform, getting started, role playbooks, feature deep dives, and operations.

PART I — THE PLATFORM

- 1** Platform Overview
- 2** Technology Stack
- 3** System Architecture
- 4** Core Concepts
- 5** Roles, Permissions & the Capability Matrix

PART II — GETTING STARTED

- 6** Signing In — Web Password & Mobile PIN
- 7** Finding Your Way Around

PART III — ROLE PLAYBOOKS

- 8** Guard (Mobile)
- 9** Supervisor (Mobile + Web)
- 10** Control Room (Web)
- 11** Manager (Web)
- 12** Administrator (Web)
- 13** Super User (Web)

PART IV — FEATURE DEEP DIVES

- 14** The Incident Lifecycle, End to End
- 15** Logging an Occurrence — Every Method
- 16** The SLA Engine & Alerts
- 17** Patrols & Checkpoint Scanning
- 18** Gate-House: Visitors & Keys
- 19** Shifts & Handovers
- 20** Tasks
- 21** Reports & PDF Export
- 22** Notifications & Deep Links

PART V — ADMINISTRATION & OPERATIONS

- 23** Integrations — Webhooks, API Tokens, Edge Functions
- 24** Deployment & Operations Runbook
- 25** Security Posture
- 26** Appendix — Accounts, Glossary, Data Model

PART I

The Platform

What DigiLog 360 is, the technology it is built on, how the pieces fit together, and the concepts — organisations, occurrences, OB numbers, SLAs, capabilities — that every other chapter builds on.

Chapter 1 — Platform Overview

Chapter 2 — Technology Stack

Chapter 3 — System Architecture

Chapter 4 — Core Concepts

Chapter 5 — Roles, Permissions & the Capability Matrix

Platform Overview

01

A multi-tenant security operations platform: control room, manager review, field guards, gate-house, patrols and supervision — across many sites and many client organisations from one deployment.

DigiLog 360 digitises the daily operations of a security company. Guards log incidents with photos and voice notes from their phones — even without signal. Patrols are verified with real QR, NFC and GPS checkpoint scans. The control room watches a live SLA board that flags breaches the moment they happen. Managers review and sign off every incident. Administrators shape the platform per organisation, and a super user runs many client companies from one deployment.

1.1 Three surfaces, one platform

WEB

Admin Console

Next.js 15 web app for the control room, managers, admins and super users. Live dashboards, occurrence explorer, reports, user & site management, platform settings.

MOBILE

Guard App

Expo / React Native app for guards and supervisors. PIN-only sign-in, occurrence logging with photos, patrol scanning, gate-house registers, shifts and handovers. Works offline.

BACKEND

Supabase Cloud

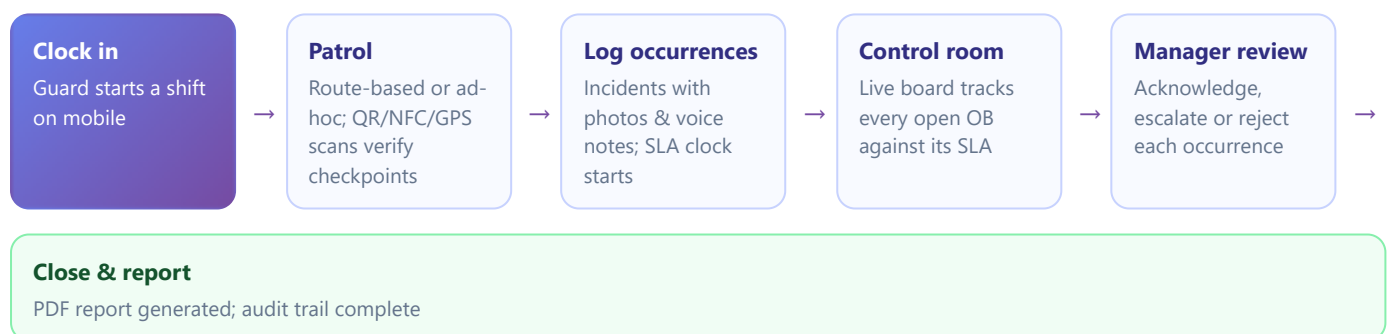
Postgres database with row-level security, authentication, file storage, realtime WebSocket streams and 15 serverless edge functions. One project hosts every tenant.

1.2 What it replaces

DigiLog 360 is a ground-up rebuild of the legacy ASP.NET MVC "OccurrenceBook" application. Everything the old system did is here, plus:

AREA	LEGACY SYSTEM	DIGILOG 360
Tenancy	One deployment per client	True multi-tenant — one deployment, many organisations, hard data isolation
Patrols	Start/end timer only	Verified checkpoint scans — QR, NFC and GPS geofence — plus schedules & late alerts
Mobile login	Email + password	Employee number + 4-digit PIN with brute-force lockout
SLA	Static report	Live realtime board, per-org SLA matrix, automatic push/email breach alerts
Connectivity	Online only	Offline queue — guards log without signal; syncs automatically
Permissions	Fixed roles	Six ranked roles, multi-role users, and a per-org capability matrix the super user edits live
Evidence	Photos only	Photos, voice notes with Whisper transcription, OCR of handwritten notes, SA licence scanning
Integration	None	Signed outbound webhooks, org-scoped API tokens, Resend email alerts with a per-recipient audit ledger, public health check

1.3 The operational day at a glance



TIP

Every arrow above is automatic. Guards don't hand anything over on paper: the moment an occurrence is saved on a phone, it is on the control-room board with its SLA countdown running, and the manager's queue updates in realtime.

Technology Stack

Modern TypeScript everywhere — Next.js 15 on the web, Expo on mobile, Supabase underneath, one shared types package binding them together.

2.1 Backend — Supabase

Component	Technology	Role in the Platform
Database	Postgres 15 (+ <code>pgcrypto</code> , <code>uuid-oss</code> , <code>pg_trgm</code>)	All domain data; row-level security enforces tenant + role isolation on every table
Auth	Supabase Auth (JWT)	Web email/password sessions, optional TOTP 2FA, magic-link minting for the mobile PIN flow
Storage	Supabase Storage	Private buckets for occurrence photos & voice notes; public bucket for org branding
Realtime	Postgres replication → WebSockets	Live boards, notification bells, guard map markers, comment threads
Serverless	Edge Functions (Deno / TypeScript)	16 functions: PIN login, user/org provisioning, task & occurrence email alerts (Resend), webhooks, SLA monitor, patrol watcher, Whisper transcription, health check
Email	Resend	Task & occurrence alert emails from a verified sending domain, with per-org branding and an immutable per-recipient delivery ledger
Scheduling	<code>pg_cron</code>	SLA monitor + patrol watcher every 5 minutes; the email-alert sweep every minute (self-scheduling — no manual setup)

2.2 Admin console — `apps/admin`

Framework

Next.js 15 App Router · React 19 · TypeScript 5.6 strict · Tailwind CSS 3.4 with dark mode. Server components fetch data in parallel batches; middleware validates auth once per navigation.

Auth & security

`@supabase/ssr` cookie sessions · custom CSP, HSTS (2 y), X-Frame-Options DENY · optional Sentry error monitoring.

UI libraries

`lucide-react` icons (tree-shaken registry) · `recharts` dashboards · `qrcode.react` printable checkpoint labels · `react-leaflet` live guard map (lazy-loaded).

Intelligence

`tesseract.js` client-side OCR pulls text out of photographed handwritten notes straight into the log-occurrence form. Minimal i18n: English, Afrikaans, isiZulu, isiXhosa.

2.3 Mobile app — `apps/mobile`

Framework

Expo SDK 52 (managed) · React Native 0.76 new architecture · `expo-router` 4 file-based navigation.

Sensors & capture

`expo-camera` QR scanning + photos · `expo-location` GPS scans + live position reporting · `expo-av` voice notes · `react-native-nfc-manager` NFC tag reads (dev build).

Resilience

`expo-secure-store` + AsyncStorage session persistence · offline occurrence queue with auto-flush and poison-row protection · `@react-native-community/netinfo` connectivity watching.

Notifications & drawing

`expo-notifications` push registration and tap deep-linking · `react-native-svg` signature canvas for handovers and charts.

2.4 Shared package & tooling

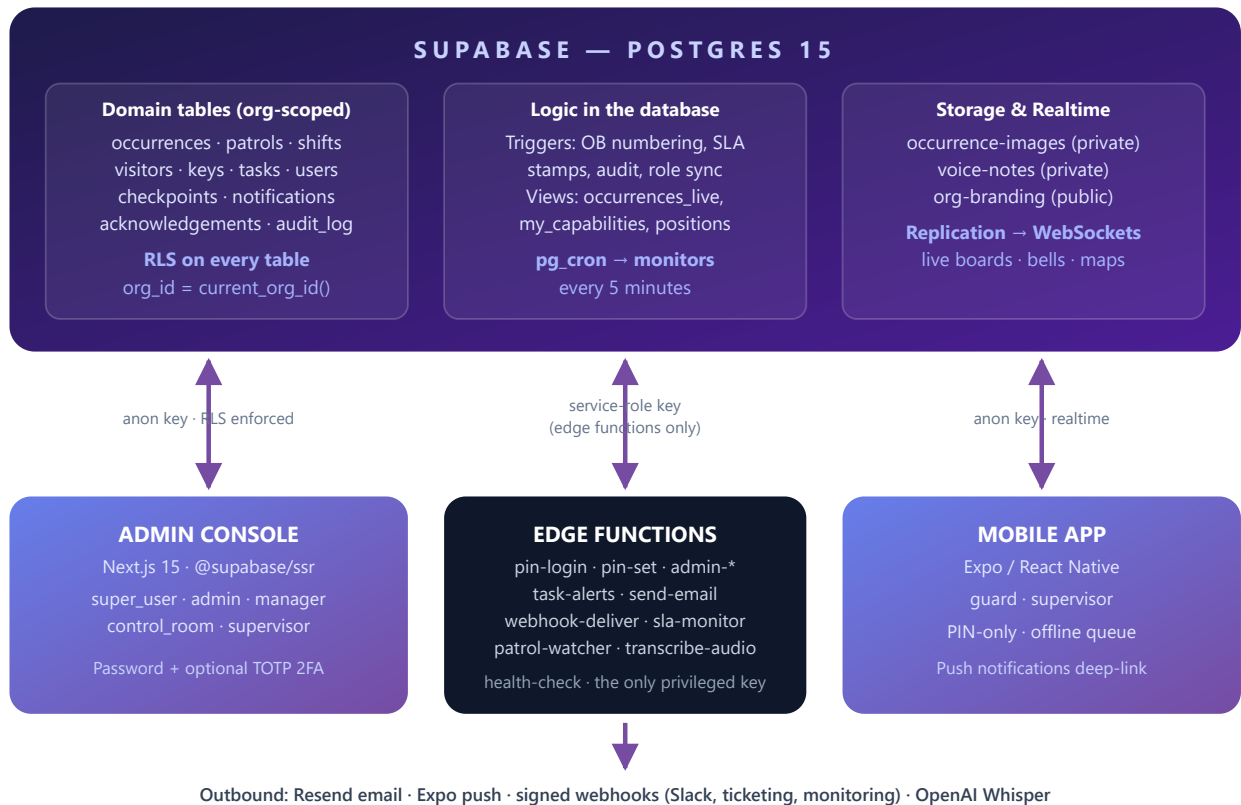
`packages/shared` (`@digilog/shared`) keeps both apps honest: database types generated from Postgres, Zod validation schemas, SLA helpers, role & capability constants and predicates, URL filter serialisers, and the i18n dictionaries. A change to a status colour or an SLA rule lands in one file and both apps pick it up.

- **Monorepo:** npm workspaces (admin + shared); mobile sits outside the workspace to avoid React dual-version conflicts.
- **CI:** GitHub Actions — typecheck of shared, admin and mobile plus an admin build smoke test on every pull request.
- **Performance:** shared First-Load JS held at ~104 kB; heavy libraries (recharts, leaflet, tesseract) are route-scoped and lazy-loaded. Composite DB indexes match every hot query path.

System Architecture

03

Three clients, one Postgres brain. Row-level security is the floor; edge functions hold the only privileged key; realtime replication keeps every screen live.



3.1 How a request flows

- **Apps talk to Postgres directly** with the public anon key. They can only ever see what row-level security allows — the guard's phone physically cannot query another organisation's rows.
- **Privileged work goes through edge functions.** Creating users, minting API tokens, verifying PINs, sending email: only these Deno functions hold the service-role key, and each one re-checks the caller's role before acting.
- **Realtime is push, not poll.** Postgres replicates changes on key tables (occurrences, comments, notifications, tasks, scans...) to WebSocket subscribers. The admin's `<RealtimeRefresh>` component debounce-refreshes any server-rendered table when its tables change.
- **Cron closes the loop.** Every 5 minutes, `sla-monitor` hunts for breached or update-due occurrences and `patrol-watcher` hunts for missed patrol slots — then notify inbox, push and email. Every minute, `task-alerts` drains the email outbox and scans for overdue tasks, so no alert email ever waits longer than ~60 seconds.

NOTE — WHY THIS IS SAFE

The capability matrix (Chapter 5) hides UI, but security never depends on it: even a modified client with a stolen anon key hits RLS, which re-evaluates the caller's org and roles on every single query. Storage is protected the same way — file paths must begin with the caller's own `<org_slug>/` prefix.

Core Concepts

04

Organisations, sites, OB numbers, the occurrence lifecycle, severities and SLAs, and the audit trail — the vocabulary of everything that follows.

4.1 Organisations & multi-tenancy

An **organisation** is a tenant — typically one client security company. Every row of domain data carries an `org_id`, and row-level security guarantees users only ever see their own organisation's data. The only exception is the `super_user`, who sees across all tenants. Each organisation gets its own branding, SLA matrix, occurrence-type taxonomy, capability grants, webhooks and API tokens.

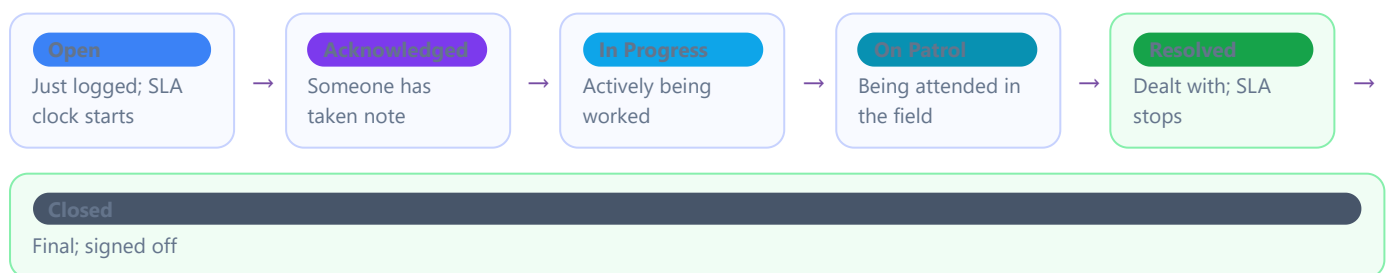
4.2 Sites

A **site** is a physical location an organisation guards (e.g. Sandton, Cape Town, Boksburg, HQ Central in the demo tenant). Guards and supervisors are anchored to a primary site; users can additionally be assigned to multiple sites (`site_ids[]`). Occurrences, patrols, shifts, visitors and keys all belong to a site, and dashboards can filter by site.

4.3 Occurrences & OB numbers

An **occurrence** is the core record: any incident or notable event. On insert, a database trigger assigns the next sequential **OB number** for the year (e.g. `OB-2026-04180`) — collision-proof even past 9999 entries, with a self-healing sequence guard after bulk imports. Each occurrence carries a type, severity, site, description, the logger, timestamps, and optionally: photos, voice notes, comments, an assigned reviewer, a report, and a manager acknowledgement.

4.4 The occurrence lifecycle



Resolved and **Closed** are terminal — reaching either stamps `closed_at` and stops the SLA countdown. Status changes are made from the web occurrence detail, the live board, or the mobile status sheet, and every change lands in the occurrence's update timeline.

4.5 Severity & the SLA matrix

SEVERITY	RESOLVE WITHIN	UPDATE EVERY	TYPICAL USE
Critical	1 hour	30 min	Armed incident, fire, medical emergency
High	4 hours	60 min	Break-in, serious damage, assault
Medium	24 hours	6 hours	Theft report, suspicious activity
Low	168 hours (7 days)	24 hours	Maintenance issue, routine note

These are the platform defaults; each organisation can override any row from [Sidebar → Administration → SLA Matrix](#). Chapter 16 covers how the countdown, the live board and the automatic alerts work.

4.6 Capabilities

Roles decide what you *may* do (the security floor, enforced in the database). **Capabilities** decide what you *see* — several dozen keys like `occurrences.log` or `patrols.scan` gate every nav item, button and mobile tab. The super user edits the role × capability grid per organisation, live, with no redeploy. Details in Chapter 5.

4.7 The audit trail

Every sensitive action — sign-ins, PIN attempts, user and role changes, org changes, capability edits, manager decisions — is written to an immutable **audit log** with actor, role, target, IP and timestamp. Admins view it at [Sidebar → Administration → Audit Log](#) (capability `audit.view`). Historical activity can be reconstructed after a legacy import; reconstructed rows stay tagged `backfilled` so they are never confused with live capture.

WARNING

Nothing in DigiLog is silently deleted. Profiles, sites, occurrences and organisations are **soft-deleted** (hidden but recoverable by engineers). The only true deletions are the super user's organisation teardown and user deletion — both deliberately loud, confirmed actions.

Roles, Permissions & the Capability Matrix

05

Six ranked roles form the security floor. A per-organisation capability grid — edited live by the super user — shapes exactly what each role sees.

5.1 The six roles

RANK	ROLE	SCOPE	PRIMARY SURFACE	IN ONE SENTENCE
100	super_user	All organisations	Admin web	Runs the platform: creates tenants, edits permissions, sees everything.
80	admin	One organisation	Admin web	Full control of their org: users, sites, branding, SLA, types, integrations.
60	manager	One organisation	Admin web	Reviews every occurrence: acknowledge, escalate or reject, plus staff reporting.
50	control_room	Site or org	Admin web	Watches the live SLA board, logs and updates occurrences, runs reports.
40	supervisor	One site	Web + mobile	Runs the site's guards: live board, team view, patrol oversight, approvals.
20	guard	One site	Mobile only	Logs occurrences, runs patrols, staffs the gate house, works shifts.

5.2 Multi-role users

A user can hold **any combination** of roles — the profile stores the full set, and a database trigger keeps the "primary" role in sync for display and default landing. Common combinations:

control_room + manager

An operator who also reviews and signs off incidents.

admin + supervisor

An org admin who covers site shifts as a supervisor.

guard + supervisor

A field officer who also runs other guards.

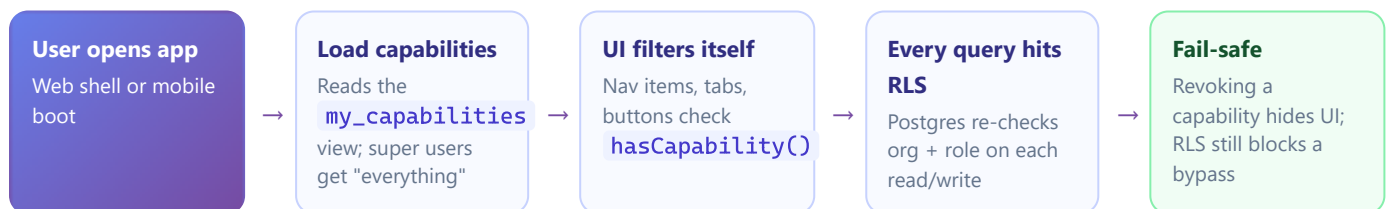
Effective permissions are the **union** of all held roles, everywhere — RLS, capabilities, and navigation.

5.3 Capability areas

Every feature maps to a capability key. The main areas (excerpt of the registry):

AREA	EXAMPLE KEYS
Dashboard	<code>dashboard.view</code> · <code>dashboard.cross_org</code>
Occurrences	<code>occurrences.view_all</code> · <code>occurrences.log</code> · <code>occurrences.update_status</code> · <code>occurrences.assign</code> · <code>occurrences.bulk_actions</code> · <code>occurrences.comment</code> · <code>occurrences.export_csv</code>
Reports	<code>reports.view</code> · <code>reports.create</code> · <code>reports.export_pdf</code>
Manager	<code>manager.acknowledge</code> · <code>manager.escalate</code> · <code>manager.reviewed_logs</code>
Patrols	<code>patrols.view</code> · <code>patrols.run</code> · <code>patrols.scan</code> · <code>patrols.end_remote</code> · <code>patrols.schedule_manage</code> · <code>checkpoints.manage</code>
Field ops	<code>team.view</code> · <code>visitors.manage</code> · <code>keys.manage</code> · <code>shifts.view_all</code> · <code>shifts.clock</code> · <code>guards.map_view</code>
Users & sites	<code>users.view</code> · <code>users.create</code> · <code>users.edit</code> · <code>users.reset_pin</code> · <code>sites.manage</code>
Settings	<code>org.edit_branding</code> · <code>org.edit_sla</code> · <code>org.edit_types</code> · <code>webhooks.manage</code> · <code>api_tokens.manage</code> · <code>audit.view</code>
Personal	<code>notifications.view_own</code> · <code>preferences.manage</code> · <code>security.manage_2fa</code>
Mobile layout	Per-role tab-bar keys, home-card container keys, on/off-duty key
Super user	<code>super.orgs_manage</code> · <code>super.users_cross_org</code> · <code>super.platform_health</code> · <code>super.permissions_manage</code>

5.4 How the two layers work together



NOTE — PRIVILEGE-ESCALATION GUARD

Database triggers block anyone but a super user from granting `super_user`, moving users between organisations, or removing a user's last role. Org admins cannot delete super users or themselves.

5.5 Auditing your own access — My Access

Any signed-in user can open [Sidebar → My Access](#) to see: the roles they hold (primary highlighted), every capability currently granted with *via {role}* provenance, and the list of pages they can actually open. If someone reports "I can't see the Patrols menu," their My Access page is the first place to look.

PART II

Getting Started

Signing in on each surface — including the mobile PIN flow and its lockout protection — and a guided tour of the admin console sidebar and the mobile app's tabs.

Chapter 6 — Signing In: Web Password & Mobile PIN

Chapter 7 — Finding Your Way Around

Signing In

06

Web users sign in with email and password (plus optional 2FA). Mobile users never touch a password — organisation, employee number and a 4-digit PIN.

6.1 Web — Admin Console

Sign in to the admin console

Browser → your DigiLog URL

- 1 Open the admin console URL in any modern browser. You land on the **login page**.
- 2 Enter your **email address** and **password**, then select **Sign in**.
- 3 If your account has **2FA enabled**, enter the 6-digit code from your authenticator app.
- 4 You arrive on your default landing page (usually the **Dashboard**). Your sidebar shows only what your roles and capabilities allow.

Who: all web roles · Sessions are HttpOnly cookies; closing the tab does not sign you out. Use the avatar menu → Sign out on shared machines.

Enable two-factor authentication (recommended)

Sidebar → Settings → Security (2FA)

- 1 Open [Security \(2FA\)](#) from the sidebar's personal section.
- 2 Select **Enrol**. A QR code appears.
- 3 Scan it with any authenticator app (Google Authenticator, Authy, 1Password...).
- 4 Type the current 6-digit code to confirm. From now on, sign-in asks for a code.

Capability: `security.manage_2fa` · Self-service for any web user.

6.2 Mobile — PIN-only sign-in

The guard app deliberately has **no password**. Identity is **organisation slug + employee number**, proven with a 4-digit PIN.

First sign-in on a phone

Mobile app → Login screen

- 1 Open the app. On first launch, tap **Switch account**.
- 2 Enter your **organisation code** (e.g. `digilog-demo`) and your **employee number** (e.g. `GRD001`). The app remembers both — you only do this once per phone.
- 3 Type your **4-digit PIN** on the keypad.
- 4 You land on **Home**, showing your name, site and quick actions.

Who: guard · supervisor (mobile roles). Your PIN is set by an admin when your account is created; you can change it in Settings.

What happens behind the keypad



TIP — LOCKED OUT?

Wait 15 minutes and try again, or ask any admin to reset your PIN ([Users](#) → : → [Reset PIN](#)). Admin resets take effect immediately.

NOTE

PINs are stored only as bcrypt hashes and are never saved on the phone. Responses are timing-normalised so attackers can't probe which employee numbers exist.

Change your PIN

Mobile → Settings → Change PIN

- 1 Open **Settings** (gear icon from Home) and choose **Change PIN**.
- 2 Enter your **current PIN**, then the **new PIN** twice. Mismatches highlight live; use the show/hide toggle if unsure.
- 3 Confirm. The change is immediate and audit-logged.

Forgot your PIN entirely? Only an admin can reset it — the app has no self-service recovery by design.

Finding Your Way Around

07

The admin console sidebar, section by section — and the mobile app's tabs, screens and quick actions.

7.1 Admin console layout

The console is a two-pane shell: a collapsible **sidebar** grouped into sections, and the content area. The header carries the **site context indicator**, the live **notification bell**, the **theme toggle** (light/dark, remembered per browser) and your **avatar menu** with sign-out. Sections and items appear only if your roles and capabilities allow them.

SECTION	ITEM	WHAT IT IS	CAPABILITY
Overview	Dashboard	KPI cards, trends, breakdowns; filterable by site	<code>dashboard.view</code>
	Live Occurrences	Realtime board of open OBs with SLA badges	—
	Notifications	In-app inbox	<code>notifications.view_own</code>
Occurrences	All Occurrences	Paginated, filterable explorer + saved views + bulk actions	<code>occurrences.view_all</code>
	Log New Occurrence	The web log-incident form (with OCR)	<code>occurrences.log</code>
	Reports	Report list + one-click PDF print	<code>reports.view</code>
	History	Closed occurrences + completed patrols	<code>occurrences.view_all</code>
	Evidence Photos	Browse every photo across all occurrences	<code>reports.view</code>
	Assigned to Me	Your personal review queue	<code>occurrences.view_assigned</code>
	Tasks	To-do board, linkable to OBs	—
Field Operations	Patrols	Active + recent patrols with route progress	<code>patrols.view</code>
	Checkpoints	QR/NFC/GPS checkpoint CRUD + printable labels	<code>checkpoints.manage</code>
	Team Status	Who's available / on patrol	<code>team.view</code>
	Visitor Log	On-site now + visit history	<code>visitors.manage</code>
	Key Register	Held vs available keys	<code>keys.manage</code>
	Shifts	Clock-in/out roll-up + KPIs	<code>shifts.view_all</code>
	Patrol Schedules	Recurring patrol expectations	<code>patrols.schedule_manage</code>
	Guard Map	Live map of on-patrol guards	<code>guards.map_view</code>
Manager	Acknowledgements	Queue of occurrences awaiting a manager decision	<code>manager.acknowledge</code>
	Reviewed Logs	Historical decisions	<code>manager.reviewed_logs</code>
	Performance Dashboard	Per-person / per-role activity roll-up	—
	Audit Log	Immutable action trail	<code>audit.view</code>
Administration	Users	User CRUD, roles, sites, PIN reset	<code>users.view</code>
	Sites	Site CRUD	<code>sites.manage</code>

	Organisation	Branding + contact details	org.edit_branding
	SLA Matrix	Per-severity SLA overrides	org.edit_sla
	Occurrence Types	Custom incident types + taxonomy	org.edit_types
Personal	My Access	Your roles, capabilities and reachable pages	—
	My Preferences	Push / email notification toggles	preferences.manage
	Security (2FA)	TOTP enrolment	security.manage_2fa
	Webhooks	Outbound integrations	webhooks.manage
	API Tokens	Org-scoped access tokens	api_tokens.manage
Super User	Organisations / All Users	Tenant + cross-org user management	super.orgs_manage · super.users_cross_org
	Permissions	Role × capability matrix editor	super.permissions_manage
	Branding · Form Builder · Mobile Layout	Per-org customisation	—
	Email Alerts	Per-org Resend email configuration, live preview, test sends + the delivery audit	—
	Taxonomy · Assignees · Mgmt Reports · Health	Platform-wide controls	super.platform_health

7.2 Mobile app layout

TAB BAR

Four tabs (configurable per role)

Home — greeting, shift card, KPIs, quick actions, supervisor section.

Patrol — route picker, active patrol status, checkpoint list.

Log — the log-occurrence form (also reachable via the floating "+").

My Logs — everything you've logged, searchable and live-updating.

SCREENS

Beyond the tabs

Scan (QR/NFC/GPS) · **Duty** (one-tap on/off patrol) · **Shift** (clock in/out) · **Handovers** · **Inbox** · **Settings** · **Tasks** ·

Gate house: Visitors & Keys · **Supervisor:** Site Board & Team · **Occurrence detail** (deep-linked from notifications).

NOTE — YOUR TABS MAY DIFFER

The super user configures the mobile tab bar and home cards per role from [Super User → Mobile Layout](#). If a tab described in this manual is missing on your phone, it has been switched off for your role — not broken.

TIP — THE PENDING-SYNC BANNER

If your phone shows a "**N pending sync**" banner on Home or the Log tab, you have occurrences saved offline that haven't reached the server yet. They flush automatically the moment signal returns — don't re-log them.

PART III

Role Playbooks

Task-by-task instructions for every role. Each task card shows where to find the feature, the numbered steps, who can do it, and the capability key that gates it.

Chapter 8 — Guard (Mobile)

Chapter 9 — Supervisor (Mobile + Web)

Chapter 10 — Control Room (Web)

Chapter 11 — Manager (Web)

Chapter 12 — Administrator (Web)

Chapter 13 — Super User (Web)

Guard — Mobile

08

Your whole working day lives on the phone: clock in, patrol, log what happens, staff the gate, hand over, clock out.

8.1 Start and end your shift

Clock in

Home → Shift card · or Quick actions → My shift

- 1 On **Home**, tap the **Shift card** (it shows "Off shift" when you're clocked out).
- 2 Tap **Clock In**. The card flips to a **live elapsed counter** (hours + minutes, ticking).
- 3 Work normally — the shift stays open in the background all day.

Capability: `shifts.clock` · Only one shift can be open at a time; the system prevents double clock-ins.

Clock out

Home → Shift card → My shift

- 1 Open your shift from the Home shift card.
- 2 Optionally type **closing notes** (anything the next shift should know).
- 3 Tap **Clock Out** and confirm. Your shift, its duration and notes appear in your recent-shifts list — and on the web Shifts roll-up.

Consider writing a **handover** (§8.6) before clocking out if there are open issues.

8.2 Go on patrol

Start a route patrol & scan checkpoints

Patrol tab

- 1 Open the **Patrol** tab. Pick a **route** (or choose ad-hoc if no route applies) and tap **Start**.
- 2 The live status card shows elapsed time and **checkpoints scanned / total**. While on patrol your phone reports its position every 30 seconds — the control room sees you on the Guard Map.
- 3 At each checkpoint tap **Scan checkpoint** and use the right mode: **QR** — point the camera at the checkpoint label · **NFC** — hold the phone against the tag · **GPS** — tap when standing at the point; the app verifies you're inside the geofence.
- 4 The route list ticks each checkpoint off as done. Missed ones stay visibly open.
- 5 When the route is complete tap **End patrol**. Duration, scans and completion are saved and visible to your supervisor instantly.

Capabilities: `patrols.run` · `patrols.scan` · Full scanning detail in Chapter 17.

Quick on-duty / off-duty toggle

Duty screen (Home quick action)

- 1 Open **Duty** from Home. One big switch: **On Patrol / Off Patrol**.
- 2 Tap to go on duty — this starts an **ad-hoc patrol** (no route, no checkpoints) with a live elapsed timer, and begins location reporting.
- 3 Tap again to go off duty and end the patrol.

Use Duty for gate posts and static guarding; use the Patrol tab when you must prove checkpoint coverage.

8.3 Log an occurrence (incident)

Log an incident from your phone

Log tab · or the floating “+” on Home

- 1 Tap the “+” **button** (or the Log tab).
- 2 Pick the **type** from the chips — your organisation's custom types come first (e.g. Intrusion, Theft, Medical, Maintenance).
- 3 Pick the **severity**: **Critical** **High** **Medium** **Low**. This sets the response clock — when in doubt, go one level higher.
- 4 Write the **description**: what, where exactly, who was involved, what you did.
- 5 Add **photos** — camera or gallery, up to 6. Evidence photos massively strengthen the record.
- 6 Optionally record a **voice note** — it is transcribed automatically and attached to the occurrence.
- 7 Tap **Submit**. You get a toast with the new **OB number** and land on the occurrence detail.

Capability: [occurrences.log](#) · The SLA clock starts the moment you submit. The control room sees it in realtime.

TIP — NO SIGNAL? LOG ANYWAY.

If the phone is offline (or the upload fails), the occurrence and its photos are saved to the device queue and submitted automatically when signal returns. The “N pending sync” banner on Home tracks this. Never wait for signal to log — the timestamp of your entry is what matters.

8.4 Track your logs

Check the status of something you logged

My Logs tab

- 1 Open **My Logs**. Every occurrence you've logged is listed, newest first.
- 2 Use the **search box** or the **All / Open / Closed** filter to find an entry.
- 3 Tap it to open the detail: status badge, description, photos, the comment thread and the full update timeline. Status changes made by the control room appear live while you watch.

You'll also get a push notification if an occurrence is assigned to you or its status changes in ways that affect you.

8.5 Gate-house duty — visitors & keys

Sign a visitor in (with licence scanning)

Home → Quick actions → Visitors → "+"

- 1 Open **Visitors**. The screen shows everyone currently on site. Tap the **"+" FAB** to sign someone in.
- 2 Fastest way — scan: tap **Scan licence disk** (vehicle) or **Scan driver's licence** (person). Point the camera at the barcode; name, ID number and vehicle registration fill themselves in.
- 3 Or type the details: **full name, ID number, company, vehicle registration, who they're visiting**.
- 4 Tap **Sign In**. The visitor appears in the on-site list — and on the web Visitor Log.

Capability: `visitors.manage` · Poor light? Use the torch toggle in the scanner, or pick a photo of the licence from the gallery.

Sign a visitor out

Visitors → On site list

- 1 **One tap:** find the visitor in the on-site list and tap **Sign out**.
- 2 **Or scan out:** tap **Scan out** at the top, scan the vehicle's licence disk as it leaves — the matching visitor is signed out automatically.
- 3 The visitor moves to **Recently signed out** with their exit time.

Scan-out is ideal at a busy vehicle gate: no scrolling, no typing, no wrong-person mistakes.

Hand out and take back keys

Home → Quick actions → Keys

- 1 Open **Keys**. Keys are grouped **Held** (out) vs **Available**.
- 2 To issue: tap an available key → **Hand over** → capture the **recipient's name and ID number** → confirm.
- 3 To take back: tap the held key → **Return**. The register updates instantly, with the full hand-over history kept per key.

Capability: `keys.manage` · The web Key Register mirrors this in realtime for the control room.

8.6 Shift handovers

Write a handover for the next shift

Home → Handovers → New

- 1 Open **Handovers** and tap **New handover**.
- 2 Write the **summary** of your shift and list any **open issues** (pending OBs, expected visitors, keys still out).
- 3 **Sign** in the signature box with your finger.
- 4 Submit. The incoming guard sees a **handover prompt** on their Home screen and must acknowledge it.

Receiving one? Home shows "Handover waiting" — open it, read it, tap **Acknowledge**. Acknowledgement is timestamped.

8.7 Stay informed

- **Inbox** (bell icon) — assignment notices, SLA alerts, patrol reminders, handover prompts. Mark read, mark all, or delete.
- **Push notifications** — tapping one takes you straight to the right screen (an assigned OB opens its detail; a late-patrol alert opens the board).
- **Tasks** — if a task is assigned to you it appears in the Tasks screen with its own update thread.

Supervisor — Mobile + Web

09

Everything a guard can do, plus live oversight of the site: the SLA board, the team view, remote patrol control and status approvals.

Supervisors inherit the full Guard playbook (Chapter 8) — clocking, patrols, logging, gate house, handovers. This chapter covers what's added on top. On mobile, a **Supervisor section** appears on Home with two extra surfaces; supervisors can also sign in to the web console.

9.1 Run the site from the Site Board

Monitor the live SLA board

Mobile → Home → Supervisor → Site Board

- 1 Open **Site Board**. Every open occurrence at your site is bucketed into three columns: **Breached** · **Update Due** · **On Track**.
- 2 The board is **realtime** — new OBs and status changes appear without refreshing. Work the Breached column first, then Update Due.
- 3 Tap any card to open the **action sheet**: full details plus a status chip grid.

The same buckets drive the control room's web board — you are both looking at the same truth.

Update an occurrence's status from the field

Site Board → tap a card · or any occurrence detail

- 1 Tap the occurrence. The bottom sheet shows the **status chip grid**: Open · Acknowledged · In Progress · On Patrol · Resolved · Closed.
- 2 Pick the new status and add **notes** (what changed, what you found, what you did).
- 3 Tap **Post**. The update lands in the occurrence timeline; the SLA update-timer resets; every open screen refreshes live.

Capability: `occurrences.update_status` · Posting regular updates is what keeps occurrences out of the "Update Due" bucket.

9.2 Manage the team

See who's on and manage patrols remotely

Mobile → Home → Supervisor → Team

- 1 Open **Team**. Each member shows a live state: **on patrol** (with progress), **on shift**, or **off**.
- 2 Need to reach someone? **Tap to call** directly from their row.
- 3 A guard forgot to end a patrol? Tap their patrol → **End patrol remotely**. The patrol closes cleanly with its real metrics.

Capabilities: `team.view` · `patrols.end_remote`

9.3 Late patrols & alerts

When a scheduled patrol slot passes unsatisfied, the **patrol watcher** sends site supervisors a push notification (*patrol.late*) and an inbox entry. Tapping it opens the Site Board. Investigate: is the guard delayed, or is the schedule wrong? Adjust from the web ([Field Operations](#) → [Patrol Schedules](#)) if the expectation itself needs to change.

9.4 Supervisors on the web console

Signing in to the web console with a supervisor account shows the operational sections scoped to **your site**: Dashboard, Live Occurrences, All Occurrences, Patrols, Team Status, Visitor Log, Key Register, Shifts and the Guard Map. Use it for the big-screen view in the guard house; use mobile in the field. The workflows match Chapter 10 (Control Room), only site-scoped.

TIP — END OF DAY

A supervisor's closing routine: Site Board empty of Breached → Team all clocked out or handed over → visitor log empty of stragglers → keys back → your own handover written. Five screens, two minutes.

Control Room — Web

10

The nerve centre: watch the live board, keep every occurrence inside its SLA, log incidents phoned in, assign reviewers, and produce the reports.

10.1 Watch the live board

Monitor open occurrences in realtime

Sidebar → Overview → Live Occurrences

- 1 Keep **Live Occurrences** open on the control-room screen. Every open OB shows as a card with type, site, severity and a **live SLA badge**.
- 2 A **flash banner** announces each new OB as guards log them — no refreshing, ever.
- 3 SLA badges animate as deadlines approach and turn red on breach. Work red first.
- 4 Use each card's **Update** action to post a status change, or **Add Report** to write the formal report.

The Dashboard (`dashboard.view`) gives the manager-level view: KPIs, monthly trend, type and severity breakdowns, top sites — filterable by site chips.

10.2 Log an occurrence from the control room

Log a phoned-in or radioed-in incident

Sidebar → Occurrences → Log New Occurrence

- 1 Open **Log New Occurrence**.
- 2 Choose the **type** (category → sub-category → type, as configured for your org), the **severity**, the **site**, and the **time of the incident** (backdate if it happened earlier).
- 3 Write the **description**. Took the report on paper first? Drag a photo of the handwritten note into the **OCR drop-zone** — the text is extracted straight into the description for you to tidy.
- 4 Optionally **assign** a reviewer immediately (see §10.4) and complete any custom fields your org has added.
- 5 Select **Submit**. The OB number is issued, the SLA clock starts, and the occurrence appears on every live surface.

Capability: `occurrences.log` · Which sections appear on this form is configured per-org by the super user's Form Builder.

10.3 Find anything — the All Occurrences explorer

Search, filter and export the occurrence book

Sidebar → Occurrences → All Occurrences

- 1 Open **All Occurrences** — the full, paginated occurrence book.
- 2 Combine filters: **free-text search** (matches OB number, type, description and logger — typo-tolerant), **status**, **severity**, **site**, **type**, **date range**, and a **patrol-only** toggle.
- 3 Every filter lives in the URL — **copy the address bar** to share the exact view with a colleague.
- 4 Use **Saved views** to name a filter set (e.g. "Sandton criticals this month"), pin it, and re-apply it in one click. Admins can share saved views org-wide.
- 5 **Export CSV** downloads the visible page for spreadsheets or client packs.

Capabilities: `occurrences.view_all` · `occurrences.export_csv` · Closed history and completed patrols live under **Occurrences** → **History**; every photo across all OBs is browsable under **Occurrences** → **Evidence Photos**.

Bulk actions on multiple occurrences

All Occurrences → select checkboxes

- 1 Tick the checkbox on two or more rows. The **bulk actions bar** appears.
- 2 Choose **Acknowledge**, **Close** (adds your note to each timeline), or **Assign** (pick one reviewer for all selected).
- 3 Confirm. Each occurrence gets its own timeline entry; nothing is merged or lost.

Capability: `occurrences.bulk_actions` · Ideal for clearing a backlog of low-severity routine entries.

10.4 Work a single occurrence

Update, assign, comment and report on an OB

Any occurrence → detail page

- 1 Open the occurrence. The detail shows severity and status badges, the details grid, description, **photo gallery**, report card, comments and the update timeline.
- 2 **Change status**: use the status control, add notes, save. The timeline and SLA timers update instantly, and if the occurrence is assigned, the reviewer and the original logger are emailed the update.
- 3 **Assign a reviewer**: in the assignment card pick a person (the list is the org's assignment allow-list). They are notified in-app, by push **and by email**, and it appears in their **Assigned to Me** queue.
- 4 **Discuss**: the comment thread is realtime — mobile and web users converse live. You can edit or delete your own comments.
- 5 **Report**: add or complete the formal report; it becomes the PDF (Chapter 21).

Capabilities: `occurrences.update_status` · `occurrences.assign` · `occurrences.comment` · `reports.create`

10.5 Oversee the field

- **Guard Map** ([Field Operations](#) → [Guard Map](#), `guards.map_view`) — live map with a marker per on-patrol guard, updating as positions arrive every 30 s, with GPS-accuracy circles.
- **Patrols** — active patrols with checkpoint progress; recent patrols with duration and completion.
- **Team Status** — who is available, on shift, on patrol right now.
- **Shifts** — the clock-in/out roll-up with KPIs; **Visitor Log** and **Key Register** — the web mirror of the gate house, live.

TIP — YOUR PERSONAL QUEUE

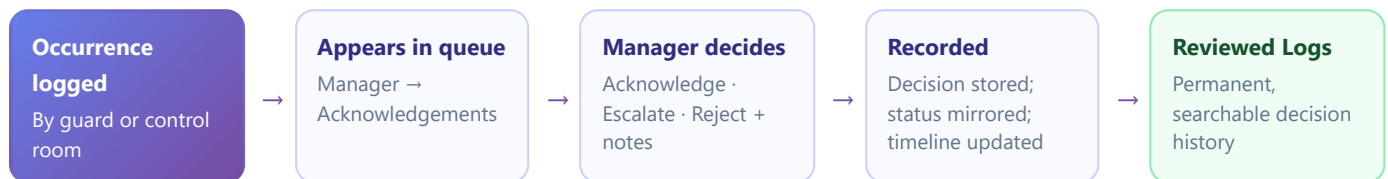
[Occurrences](#) → [Assigned to Me](#) is your own work list: everything currently assigned to you, live-updating. Clear it before end of shift.

Manager — Web

11

Every occurrence passes across the manager's desk: acknowledge it, escalate it, or reject it — with a complete reviewed history and staff performance reporting.

11.1 The review pipeline



Review and sign off occurrences

Sidebar → Manager → Acknowledgements

- 1 Open **Acknowledgements** — the queue of every occurrence not yet reviewed by a manager.
- 2 Open an entry to read the full record: description, photos, updates, comments.
- 3 Select your decision: **Acknowledge** — reviewed, handled appropriately · **Escalate** — needs further action or senior attention · **Reject** — incorrectly logged or invalid.
- 4 Add your **notes** (mandatory context for escalations and rejections) and confirm.
- 5 The decision is stored with your name and timestamp, mirrored onto the occurrence status, dropped into its timeline, and audit-logged.

Capabilities: `manager.acknowledge` · `manager.escalate` · Bulk sign-off: select multiple routine entries and acknowledge them in one action — they collapse into a single grouped entry in Reviewed Logs.

11.2 The reviewed history

Look up past decisions

Sidebar → Manager → Reviewed Logs

- 1 Open **Reviewed Logs** — every historical decision, newest first.
- 2 Each entry shows the OB, the decision (`Acknowledged` / `Escalated` / `Rejected`), the deciding manager, notes and timestamp.
- 3 Bulk sign-offs appear as one grouped entry — expand to see the individual OBs.

Capability: `manager.reviewed_logs` · This is the record you'll reach for in client meetings and disputes.

11.3 Staff performance

Run the performance dashboard

Sidebar → Manager → Performance Dashboard

- 1 Open the **Performance Dashboard** (staff reports).
- 2 Review activity rolled up **per person and per role**: occurrences logged, updates posted, patrols completed, decisions made.
- 3 **Export CSV** for payroll queries, disciplinary records or client SLAs.

Combine with the **Audit Log** ([audit.view](#)) when you need the exact who-did-what-when trail behind a number.

NOTE — MANAGERS AND SLA ALERTS

Managers receive breach notifications (inbox, push and email summaries, according to their preferences under [Settings → My Preferences](#)). A breached critical at 02:00 will find you — that is by design.

Administrator — Web

12

Full control of one organisation: people, sites, branding, SLAs, incident types, notification preferences and integrations.

12.1 Manage people

Create a user

Sidebar → Administration → Users → New user

- 1 Open **Users** and select **New user**.
- 2 Enter **full name** and **email** (email is the web sign-in identity and must be unique platform-wide).
- 3 Tick the **roles** in the checkbox grid — any combination up to your own rank. Set the **job title** for display.
- 4 Assign the **primary site** and, if needed, additional sites (multi-site assignment).
- 5 For mobile users (guard / supervisor): set the **employee number** (unique in your org) and an initial **4-digit PIN**.
- 6 For web users: set the initial **password**.
- 7 Save. The account is live immediately; tell the user their credentials out-of-band.

Capabilities: `users.create` · `users.edit` · Org admins cannot grant **super_user** or move users to another organisation — those are super-user actions.

Edit roles, reset a PIN, deactivate or delete

Users → row actions

- 1 **Edit:** open the user, adjust roles / sites / job title / employee number, save. Changes apply on their next page load.
- 2 **Reset PIN:** choose **Reset PIN**, enter a new 4-digit PIN, confirm. Use this when a guard is locked out or forgot theirs.
- 3 **Deactivate:** soft-disable the account (kept for history, cannot sign in).
- 4 **Delete:** permanently removes the account and profile. You cannot delete yourself, and org admins cannot delete super users. Type-to-confirm.

Capabilities: `users.reset_pin` · `users.deactivate` · Every one of these actions is audit-logged. Users list exports to CSV.

12.2 Manage sites

Add or edit a site

Sidebar → Administration → Sites

- 1 Open **Sites** and select **New site** (or edit an existing row).
- 2 Enter the name, address and contact details. Save.
- 3 Assign users to the site from the **Users** page; create its **checkpoints** and **patrol routes** from Field Operations (Chapter 17).

Capability: `sites.manage` · Sites are soft-deleted — history at a closed site stays intact.

12.3 Shape the platform for your organisation

Tune the SLA matrix

Sidebar → Administration → SLA Matrix

- 1 Open **SLA Matrix**. Each severity row shows the current **resolve-within** and **update-every** targets.
- 2 Edit a row (e.g. Critical: resolve 45 min, update 15 min for a high-risk client) and save.
- 3 New occurrences pick the new targets up immediately; each row has **reset to default**.

Capability: `org.edit_sla` · Defaults: Critical 1 h/30 m · High 4 h/60 m · Medium 24 h/6 h · Low 7 d/24 h.

Manage occurrence types & custom fields

Sidebar → Administration → Occurrence Types

- 1 Open **Occurrence Types**. Types are organised as **Category** → **Sub-category** → **Type**.
- 2 Add a type: pick its category, name it, set a **default severity**, and optionally define **custom fields** (extra inputs that appear on the log form for this type).
- 3 Disable types you no longer want offered — history keeps them for old records.
- 4 Custom types appear **first** in the mobile app's type chips and the web form's pickers.

Capability: `org.edit_types` · Renames update historical records too, so reports stay grouped.

Branding & organisation details

Sidebar → Administration → Organisation

- 1 Open **Organisation**. Update the display name, contact details and **logo** (uploaded to the branding bucket; appears on the console and PDF reports).
- 2 Save. The plan field is read-only for org admins.

Capability: `org.edit_branding`

12.4 Watch the audit log

Administration → **Audit Log** ([audit.view](#)) shows every sensitive action: colour-coded action chips, actor name and role, the affected record, IP address and timestamp. Filter by action or actor when investigating. Rows reconstructed from legacy imports are tagged *backfilled*.

12.5 Integrations (summary)

Admins also own **Webhooks** ([webhooks.manage](#)) and **API Tokens** ([api_tokens.manage](#)) — pushing DigiLog events into Slack/ticketing/monitoring and letting client systems query the API. Full instructions in Chapter 23.

WARNING — LEAST PRIVILEGE

Resist granting admin to solve a visibility problem. If a manager can't see a page, the fix is almost always a **capability grant** (ask the super user), not a role upgrade. Check the user's My Access page first.

Super User — Web

13

The platform operator: create and delete whole organisations, manage users across all tenants, edit every org's permission grid, and customise each org's forms, branding and mobile layout.

13.1 Tenant management

Onboard a new client organisation

Sidebar → Super User → Organisations → New

- 1 Open **Organisations**. Each tenant shows live counts (users, sites, occurrences).
- 2 Select **New organisation**. Enter the name and a **slug** (short code, e.g. `acme-security` — it becomes part of mobile sign-in and storage paths; choose carefully, it's permanent).
- 3 Optionally bootstrap the **first admin** — name, email, password — in the same step.
- 4 Create. Default role × capability grants are seeded automatically; the org is ready for its admin to add sites and users.

Capability: `super.orgs_manage` · Follow-up: check the new org's Permissions grid, Form Builder and Mobile Layout so its roles see exactly what the client bought.

Delete an organisation

Organisations → row → Delete

- 1 Select **Delete** on the tenant row.
- 2 **Type the organisation's name** to confirm — this is deliberate friction.
- 3 Confirm. Every row of the org's data is cascade-deleted and all of its sign-in accounts are removed. No orphans remain.

This is irreversible and is the platform's only true mass deletion. You cannot delete the organisation you belong to. Export anything the client is owed first.

13.2 Cross-org users

Create or manage a user in any organisation

Sidebar → Super User → All Users

- 1 Open **All Users** — every account across every tenant, with an org column.
- 2 To create: select **New user, pick the organisation first** (the site list then filters to that org), then proceed as a normal user creation (roles, sites, PIN or password).
- 3 Edit, reset PINs, or delete any user — including granting **super_user**, which only you can do.

Capability: `super.users_cross_org` · Org membership is set explicitly per user — it is never inferred from the site.

13.3 The permission grid

Edit what each role can see, per organisation

Sidebar → Super User → Permissions

- 1 Open **Permissions** and pick the **organisation**.
- 2 The grid shows every **capability key** (rows, grouped by area) against the six **roles** (columns).
- 3 Toggle cells to grant or revoke. Changes take effect on the users' next page load — no redeploy, no restart.
- 4 Add **custom capability keys** if you need new gates for future features; built-in keys cannot be removed.

Capability: `super.permissions_manage` · Remember: this shapes the UI. The database's RLS floor cannot be widened from here — a guard can never be granted cross-org reads.

13.4 Per-org customisation

Form Builder

[Super User → Form Builder](#) — toggle which sections of the Log-Occurrence form each org sees (photos, voice notes, custom fields, assignment...). Keeps small clients' forms simple.

Mobile Layout

[Super User → Mobile Layout](#) — configure the mobile tab bar and Home cards per role per org. Backed by capability keys, applied live to phones without an app update.

Branding

[Super User → Branding](#) — toggle the parent-company (Netstream) logo per organisation, alongside each org's own logo settings.

Incident Taxonomy

[Super User → Incident Taxonomy](#) — maintain the Category → Sub-category → Type dropdown lists used across the app. Renames update historical records so reporting stays grouped.

Assignment Allow-list

[Super User](#) → [Assignment Allow-list](#) — choose exactly which profiles appear in the "Assign to" dropdowns. Keeps assignment lists short and correct.

Management Reports Access

[Super User](#) → [Management Reports Access](#) — grant specific users the management-report capability (`occurrences.log_management_report`).

13.5 Email alerts

Configure an organisation's email alerts

[Sidebar](#) → [Super User](#) → [Email Alerts](#)

- 1 Pick the **organisation**. Everything on this page is configured per tenant.
- 2 Set the identity: **master switch**, **sender name** (address comes from the platform's verified Resend domain), **reply-to**, **accent colour**, **footer note**, and whether org admins are **copied on overdue breaches**.
- 3 Tune each of the six events — **Task assigned**, **Occurrence assigned**, **Task updated**, **Occurrence updated**, **Task completed**, **Task overdue (breach)**: toggle it, and optionally override the **subject** and **intro paragraph** using `{{variable}}` chips (task title, OB number, assignee, due date...).
- 4 Watch the **live preview** — it is rendered by the exact template module the sending function uses, so what you see is what recipients get.
- 5 **Save**. New emails use the configuration immediately.
- 6 **Send a test** to any address (or blank for your own) to see the real email land in a real inbox.

Who receives what: assignment emails always go to the assignee (even self-assigned); updates and completions go to assigner + assignee minus whoever made the change; overdue goes to both plus optional admins. Per-user opt-outs under [Settings](#) → [My Preferences](#) are always respected.

Audit every outgoing email — Delivery history

[Email Alerts](#) → [Delivery history](#)

- 1 Scroll to **Delivery history** — one row per recipient per send attempt, newest first (latest 200).
- 2 Filter with the **All / Sent / Failed / Dev / Tests** chips or **search** by recipient, subject or OB number.
- 3 Every successful row carries the **Resend message ID** — proof of hand-off you can trace in the Resend dashboard. Failed rows show the exact provider error.
- 4 An amber "**N queued**" badge means emails are waiting in the outbox — the every-minute sweep sends them; press **Refresh** to watch them land.

Backed by the immutable `email_log` ledger. Test sends are recorded too, tagged so they never muddy the real history.

13.6 Platform health

[Super User](#) → [Platform Health](#) (`super.platform_health`) aggregates KPIs and recent activity across every tenant — your first stop each morning and your dashboard during incidents. Pair it with the public `health-check` endpoint wired into external uptime monitoring (Chapter 24).

NOTE — THE SUPER USER'S GOLDEN RULE

Work as `super_user` only for platform tasks. For day-to-day operational work inside a tenant, use a normal admin account in that org — it keeps the audit log honest and prevents accidental cross-tenant changes.

PART IV

Feature Deep Dives

The complete mechanics behind each feature — how an incident travels from a guard's phone to a signed-off PDF, how the SLA engine decides who gets woken up, and how every scan, visitor and key is verified and recorded.

Chapter 14 — The Incident Lifecycle, End to End

Chapter 15 — Logging an Occurrence: Every Method

Chapter 16 — The SLA Engine & Alerts

Chapter 17 — Patrols & Checkpoint Scanning

Chapter 18 — Gate-House: Visitors & Keys

Chapter 19 — Shifts & Handovers · **20** — Tasks · **21** — Reports · **22** — Notifications

The Incident Lifecycle, End to End

14

One incident, five hands, zero paper: the full journey from the moment something happens on site to the archived PDF report.



14.1 Who touches what, when

STAGE	ACTOR	SURFACE	TYPICAL TIME
Log	Guard (or control room)	Mobile Log tab / web form	Under 2 minutes, at the scene
Triage	Control room	Live Occurrences board	Within minutes — the board flashes
Work & update	Control room · supervisor · assignee	Detail page / mobile status sheet	Until resolved, updating per the SLA cadence
Review	Manager	Acknowledgements queue	Same or next business day
Report	Control room / manager	Reports → PDF	On demand, forever

NOTE — NOTHING IS LOST IN BETWEEN

Every hop above is a database write with an audit trace, an update-timeline entry, or both. If a dispute arises months later, the occurrence detail replays the entire story: who logged it, every status change with notes, every comment, the manager's decision, and when it closed.

Logging an Occurrence — Every Method

15

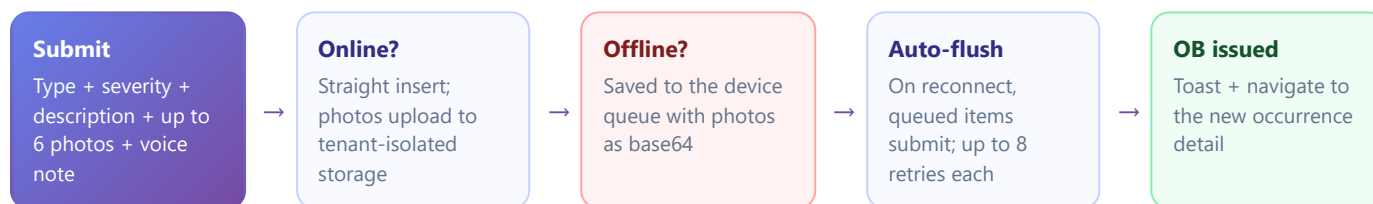
Five ways evidence gets into the book: mobile form, offline queue, web form, OCR of handwritten notes, and transcribed voice notes.

15.1 Choosing severity honestly

Severity is the single most important field — it sets the response clock and decides who gets alerted. Use the table in §4.5. Two rules of thumb: **if people are at risk, it's Critical**; if you hesitate between two levels, pick the higher one. Control room and managers can adjust severity later (the SLA recalculates automatically), but the first minutes run on your call.

15.2 The mobile path (primary)

Chapter 8.3 has the step-by-step. What the app does with your submission:



WARNING

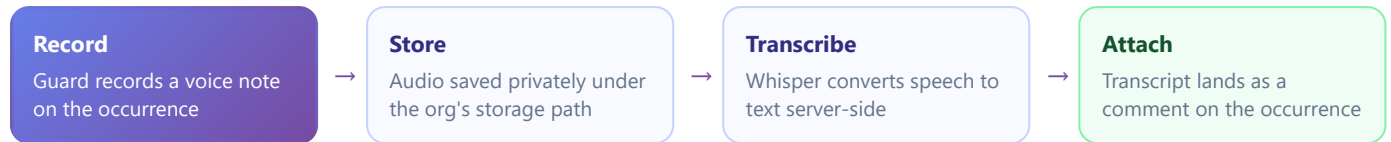
The pending-sync banner means entries are still on the phone. Don't clear the app's storage, don't uninstall, and get the phone online before end of shift — the queue survives app restarts but not a wipe.

15.3 The web path

Chapter 10.2 has the step-by-step. Web extras worth knowing:

- **OCR drop-zone** — drop a photo of a handwritten pocket-book note; Tesseract reads it into the description field, in the browser (nothing leaves your machine for OCR).
- **Backdating** — the "incident at" time is editable, so phoned-in reports carry the true incident time, while the log time is recorded separately.
- **Custom fields & form sections** — what appears is configured per organisation (admin's custom fields per type + super user's Form Builder).
- **Immediate assignment** — assign a reviewer as you log; they're notified before you've put the phone down.

15.4 Voice notes & transcription



Voice is the fastest capture under pressure — talk through what you see, and the words become searchable text on the record.

The SLA Engine & Alerts

16

Two clocks per occurrence — resolve-by and update-every — computed in the database, watched every five minutes, and escalated automatically.

16.1 Two clocks

🕒 Resolve-by clock

Set from severity at logging time (`sla_due_at`). If the occurrence isn't **Resolved** / **Closed** by then, it is **breached** — red on every board, alert storm follows.

🔄 Update-every clock

Resets on every posted update. If no one has touched the occurrence within its update interval, it becomes **update due** — amber on the boards, gentler notifications.

16.2 How the countdown works

- A database trigger stamps the SLA on every insert and recomputes it if **severity changes** (upgrade a Medium to Critical and the due time tightens instantly).
- Per-organisation overrides ([Administration](#) → [SLA Matrix](#)) take precedence over the platform defaults.
- The live view exposes `is_sla_breached`, `is_sla_update_due` and a friendly `minutes_remaining` — every board, badge and bucket reads from this one source of truth.
- Terminal statuses stop the clocks and stamp `closed_at`.

16.3 The five-minute watchdog



TIP — STOP THE AMBER TIDE

Update-due warnings are cheap to prevent: a one-line status note ("still with SAPS, awaiting case number") resets the update clock. Teach the habit; the boards stay green.

Patrols & Checkpoint Scanning

17

Provable presence: QR, NFC and GPS-verified checkpoint scans, recurring schedules with late alerts, and a live map of everyone on patrol.

17.1 Building blocks (admin setup)

Create checkpoints and routes

Sidebar → Field Operations → Checkpoints

- 1 Create a **checkpoint** per physical point (gate, generator, roof door...). Each gets an auto-generated **QR token**; optionally add an **NFC tag ID** and/or **GPS coordinates with a geofence radius** in metres.
- 2 **Print the QR labels** straight from the page and mount them at the points.
- 3 Group checkpoints into **patrol routes** in walking order.
- 4 Optionally add **patrol schedules** ([Field Operations](#) → [Patrol Schedules](#)): "Route A every 2 hours, 18:00–06:00". The system then expects patrols and alerts when one is missed.

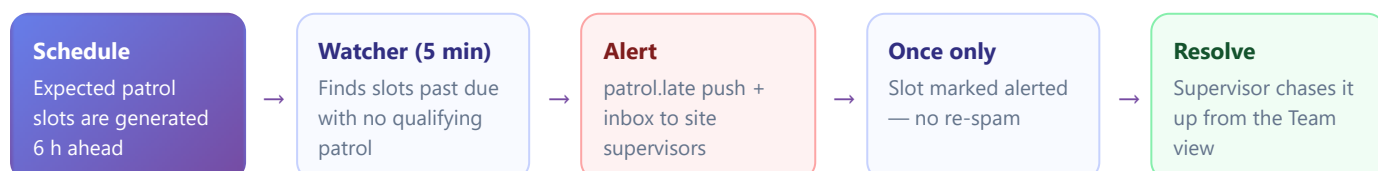
Capabilities: `checkpoints.manage` · `patrols.schedule_manage`

17.2 The three scan methods

METHOD	GUARD DOES	SYSTEM VERIFIES	BEST FOR
QR	Points camera at the label	Decodes the DIGILOG-CP: token → checkpoint match	Default everywhere; cheap labels
NFC	Holds phone on the tag	Tag ID matched (case- and colon-insensitive)	Dark or weather-exposed points; needs the dev build (not Expo Go)
GPS	Taps "scan" standing at the point	Haversine distance to the nearest checkpoint ≤ its geofence radius → verified flag	Open areas, vehicle patrols, no hardware at all

Every scan writes the method, coordinates, distance and verified flag. A patrol's progress counter climbs with each scan and is visible live on the web Patrols page and the supervisor's Team view.

17.3 Late-patrol alerts



17.4 The live guard map

Field Operations → Guard Map (`guards.map_view`) plots every on-patrol guard on an OpenStreetMap view. Positions arrive every 30 seconds while a patrol is active (and only then — off patrol, phones report nothing). Markers animate as new positions land; each shows the guard's name, site and GPS accuracy circle.

NOTE — PRIVACY BY DESIGN

Location reporting is tied strictly to an active patrol. Clocking in to a shift does not report location; only going on patrol does, and it stops the moment the patrol ends.

Gate-House: Visitors & Keys

18

A digital visitor register with SA licence scanning and scan-to-sign-out, and a key register with named, ID-verified hand-overs.

18.1 Visitor register

The mobile **Visitors** screen (guard/supervisor, capability `visitors.manage`) is the gate house's front door; the web **Field Operations** → **Visitor Log** mirrors it live for the control room. Step-by-steps are in §8.5 — here's the machinery:

Licence scanning

The sign-in sheet scans two South African documents with the phone camera: the **vehicle licence disk** (windscreen barcode → registration, make, colour) and the **driver's licence** (PDF417 barcode → full name, ID number). Both fill the form automatically — typo-free identity capture in seconds, with a torch toggle and a gallery fallback for damaged barcodes.

Scan-to-sign-out

At exit, the guard taps **Scan out** and scans the leaving vehicle's licence disk. The platform matches the registration against the on-site list and signs out the right visitor — no scrolling, no mistaken identity at a busy gate.

What's recorded

Full name, ID number, company, vehicle registration, person visited, sign-in and sign-out timestamps, and the signing guard. "Currently on site" is always accurate to the second.

Evacuation roll call

The on-site list *is* the roll call. During an evacuation, the supervisor opens Visitors (mobile or web) and reads out exactly who should be at the muster point.

18.2 Key register

- Keys are registered per site by the admin; each shows **Held** (with holder) or **Available**.
- Every hand-over captures the **recipient's name and ID number** — accountability is personal, not "someone took it".
- Returns close the loop; the complete hand-over history is kept per key.
- Web mirror at **Field Operations** → **Key Register** (`keys.manage`) — the control room sees at a glance which keys are out overnight.

Shifts & Handovers

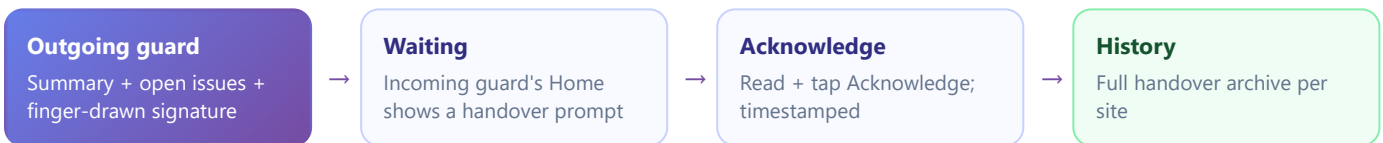
19

Clock-ins with live counters and one-open-shift enforcement, plus signed, acknowledged shift handovers.

19.1 Shifts

- **Mobile:** the Home shift card clocks in/out (§8.1); the Shift screen shows the live elapsed counter, closing notes and recent shifts with computed durations.
- **One open shift per user** is enforced by the database — a forgotten clock-out can't double up.
- **Web:** [Field Operations](#) → [Shifts](#) (`shifts.view_all`) rolls up everyone's shifts with a KPI strip — who's on now, hours this week, per site.

19.2 Handovers



Handover discipline is what stops the 06:00/18:00 information gap. The signature and acknowledgement make it a two-party record: one person handed over, one person accepted — provable later.

Tasks

20

A lightweight to-do tracker that keeps follow-up work attached to the occurrence that caused it.

Create and track a task

Web: Sidebar → Occurrences → Tasks · Mobile: Tasks screen

- 1 Open **Tasks** and select **New task**.
- 2 Title and describe it; optionally **link an OB number** (e.g. "Replace beam sensor — OB-2026-04181") and **assign** it to a user.
- 3 The assignee gets a realtime toast and inbox entry; the board self-updates for everyone.
- 4 Progress is tracked through the task's own **update thread**; close it when done.

Tasks stop "we'll fix it tomorrow" from evaporating: the occurrence closes, the follow-up survives with its own owner and trail.

NOTE — TASKS EMAIL PEOPLE AUTOMATICALLY

Assignment, status updates, completion and overdue breaches each send a branded email to the task's assigner and assignee (see §22 for exact routing). Give a task a **due date** and the platform watches it: pass the deadline and a red overdue email fires once — editing the due date re-arms it.

Reports & PDF Export

21

Every occurrence can carry a formal report, rendered to a clean A4 PDF on demand — plus CSV exports wherever there's a table.

Write and print an occurrence report

Occurrence detail → Report · or Sidebar → Occurrences → Reports

- 1 A **report shell** is auto-created when an occurrence is logged. Open the occurrence (or the Reports list) and complete the report text — the formal narrative, actions taken, outcome.
- 2 From the **Reports** list, select the print action for the OB.
- 3 A print-styled A4 page opens (org branding included) and triggers the browser's print dialog automatically — **Save as PDF** or print to paper.

Capabilities: `reports.view` · `reports.create` · `reports.export_pdf` · Management-report logging is separately gated (`occurrences.log_management_report`) and granted per user by the super user.

CSV exports

All Occurrences (visible page), Users, and the manager's staff reports all export CSV for spreadsheets and client packs. For scheduled/automated exports, use an API token (Chapter 23) instead of manual CSV pulls.

Notifications & Deep Links

22

Three channels — in-app inbox, push, email — all respecting per-user preferences, with push taps landing exactly where the work is.

CHANNEL	DRIVEN BY	USER CONTROL
In-app inbox (web bell + mobile inbox)	SLA monitor, patrol watcher, assignments, manager decisions, handovers, system notices	Mark read / mark all / delete
Push (mobile)	Same events, sent via Expo Push	Settings → My Preferences — push toggle
Email (Resend)	Task & occurrence alerts — assigned / updated / completed / overdue — plus SLA breach summaries	My Preferences — email + assignment toggles

Where a tapped push takes you

NOTIFICATION	OPENS
Anything with an occurrence (assignment, SLA, manager decision)	That occurrence's detail screen
Late patrol	Supervisor Site Board
Handover waiting	Handovers screen
Site-level SLA alert	Supervisor Site Board
Everything else	Inbox

Deep links work from cold start too — if the app was closed, it boots straight into the right screen after PIN entry.

Email alerts in depth

Every task and occurrence email is a branded, mobile-friendly message: gradient header with the organisation's name, event pill, personal greeting, a detail card with priority/severity and status chips, due time, linked OB, and the latest note.

EVERY EMAIL OPENS THE RECORD IN ONE CLICK

Each message carries a gradient **Open occurrence OB-...** (or **Open task**) button that deep-links straight into the console. The **OB number itself is a clickable link**, task emails tied to an incident carry a second **View occurrence** link, and a plain copy-paste URL sits underneath for mail clients that strip styled buttons. The plain-text version of every email carries the same links.

WARNING — LINKS NEED THE APP URL

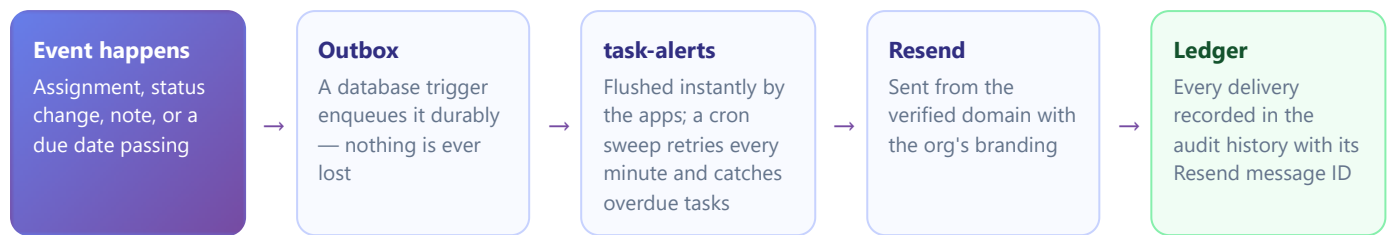
The buttons are built from the `PUBLIC_APP_URL` secret. If it is ever unset, emails still send but render **with no links at all**. Set it whenever the console moves: `supabase secrets set PUBLIC_APP_URL=https://...`

Who receives which email

EVENT	RECIPIENTS
Task assigned	The assignee — always, even when they assigned it to themselves (the email is the record)
Occurrence assigned	The reviewer, from <i>any</i> assignment path: the log form, the detail-page assignment card, or a bulk action
Task updated	Assigner + assignee, excluding whoever made the change
Occurrence updated	The assigned reviewer + whoever logged it, excluding whoever made the change. Fires on every status change and posted note — including resolution. Only <i>assigned</i> occurrences send these, so bulk actions on unassigned history stay silent
Task completed	Assigner + assignee, excluding whoever completed it
Task overdue (breach)	Assignee + assigner, plus every org admin if enabled

Per-user preferences always win: no email address, email notifications off, or (for assignments) notify-on-assignment off — no email.

How an alert email travels



TIP — CUSTOMISING AND AUDITING

The super user shapes all of this per organisation — subjects, intro copy, colours, sender identity, per-event switches — from [Super User → Email Alerts](#) (\$13.5), which also hosts the full **Delivery history** audit of every mail that went out.

PART V

Administration & Operations

For the people who run the platform itself: connecting external systems, deploying and operating the stack, the security model in depth, and reference material.

Chapter 23 — Integrations: Webhooks, API Tokens, Edge Functions

Chapter 24 — Deployment & Operations Runbook

Chapter 25 — Security Posture

Chapter 26 — Appendix: Accounts, Glossary, Data Model

Integrations

23

Signed outbound webhooks, org-scoped API tokens, Resend email delivery, and the sixteen edge functions that do the platform's privileged work.

23.1 Outbound webhooks

Push DigiLog events to Slack, ticketing or monitoring

Sidebar → Settings → Webhooks

- 1 Open **Webhooks** and select **New webhook**.
- 2 Enter the destination **URL**, choose the **events** to subscribe to, and set a **secret**.
- 3 Save. Deliveries begin immediately. Each carries an **X-DigiLog-Signature** header — an HMAC-SHA256 of the payload with your secret — so the receiver can verify authenticity.
- 4 Watch the **last-status badge** per webhook and the delivery history to catch failing endpoints.

Capability: `webhooks.manage` · Receivers should reject any payload whose signature doesn't verify.

23.2 API tokens

Mint a token for a client system

Sidebar → Settings → API Tokens

- 1 Open **API Tokens** → **New token**. Name it after the consuming system and pick its scopes.
- 2 The plaintext token is shown **exactly once** — copy it into the client system's secret store now.
- 3 Only a SHA-256 hash is kept at rest. Lost the plaintext? Revoke and mint a fresh one.
- 4 **Revoke** immediately when an integration is retired or a leak is suspected.

Capability: `api_tokens.manage` · Tokens are org-scoped — they can never read another tenant's data.

23.3 Edge functions reference

FUNCTION	PURPOSE	WHO MAY CALL
<code>pin-login</code>	Verify PIN, mint session, enforce lockout	Public POST
<code>pin-set</code>	Set / reset a PIN (self or admin)	Signed-in
<code>admin-create-user</code> / <code>admin-update-user</code> / <code>admin-delete-user</code>	User provisioning & mutation	admin · super_user
<code>admin-create-org</code> / <code>admin-delete-org</code>	Tenant provisioning & teardown	super_user
<code>admin-api-token</code>	Mint / revoke API tokens	admin · super_user
<code>admin-role-capability</code>	Grant/revoke capability matrix cells	super_user
<code>send-email</code>	Resend wrapper (logs to console if unconfigured)	Internal · admin
<code>webhook-deliver</code>	Fan out signed events, record deliveries	Internal · admin
<code>sla-monitor</code> / <code>patrol-watcher</code>	The five-minute watchdogs (Chapter 16, 17)	Cron
<code>task-alerts</code>	Drains the email outbox through Resend (task & occurrence alerts), scans for overdue tasks, sends configurable test emails, and writes the <code>email_log</code> delivery ledger	Signed-in (flush) · Cron · super_user (test)
<code>transcribe-audio</code>	Whisper transcription of voice notes	Signed-in
<code>health-check</code>	Public uptime probe (DB connectivity + counts)	Anyone

Deployment & Operations Runbook

24

From an empty Supabase project to a running platform — and the routines that keep it healthy. The full runbook lives in `PRODUCTION.md`; this is the map.

24.1 First deployment

Stand the platform up

Repo root, PowerShell

- 1 **Provision Supabase** — create a project; note the URL, anon key and service-role key.
- 2 **Configure env files** — `.env` (service role, for scripts), `apps/admin/.env.local` and `apps/mobile/.env` (URL + anon key each).
- 3 **Apply migrations** — `npx supabase db push` (or paste `supabase/_deploy_all.sql` into the SQL editor). All 45+ migrations are idempotent and safe to re-run.
- 4 **Regenerate types** — `npm run db:types`.
- 5 **Deploy the 16 edge functions** — `npx supabase functions deploy ...` (list in README §Quick Start).
- 6 **Set email secrets** — `supabase secrets set RESEND_API_KEY=... "EMAIL_FROM=DigiLog 360 <alerts@your-verified-domain>" PUBLIC_APP_URL=https://your-console` (verify the sending domain at resend.com/domains first). **PUBLIC_APP_URL is what puts the clickable buttons in every alert email** — without it they render link-free.
- 7 **Schedule cron** — SLA monitor and patrol watcher every 5 minutes (`supabase/schedule_sla_monitor.sql`). The email sweep schedules itself: call `admin_schedule_task_alerts(url, service_key)` once via RPC.
- 8 **Seed** — `node --env-file=.env scripts/seed-accounts.mjs` creates the demo org, the full role line-up and a cross-org super user.
- 9 **Host** — admin console to Vercel (root `apps/admin`); mobile via Expo EAS builds for iOS + Android.

Verify: sign in as each demo role, log a test occurrence on mobile, watch it appear on the web live board, and hit `/functions/v1/health-check`.

24.2 Routine operations

CADENCE	TASK
Daily	Glance at Platform Health (super user) and your uptime monitor on <code>health-check</code>
Weekly	Review webhook delivery failures and the email Delivery history for failed sends; check the audit log for anomalies (PIN lockouts, off-hours admin actions)
Quarterly	Rotate <code>SUPABASE_SERVICE_ROLE_KEY</code> (90 d); review user lists per org — deactivate leavers, reset their PINs
Twice yearly	Rotate anon key, <code>RESEND_API_KEY</code> , <code>OPENAI_API_KEY</code> , <code>INTERNAL_FN_KEY</code> (180 d)
On staff exit	Deactivate the account and reset/park its PIN the same day

24.3 Data import & maintenance scripts

SCRIPT (<code>SCRIPTS/...</code>)	USE
<code>seed-accounts.mjs</code>	Bootstrap super user + demo org + role line-up
<code>import-occurrences.mjs</code> · <code>import-legacy-csv.mjs</code> · <code>import-legacy-azure.mjs</code>	Bring historical occurrences in from CSV or directly from a legacy Azure SQL BACPAC
<code>fix-ob-sequence.mjs</code>	Repair the OB-number sequence after a bulk import
<code>backfill-audit.mjs</code>	Reconstruct audit history from real record timestamps (dry-run by default; rows tagged <i>backfilled</i>)
<code>verify-db-state.mjs</code> · <code>check-state.mjs</code> · <code>probe-rls-funcs.mjs</code>	Diagnostics: schema state, RLS helper behaviour

All scripts run with `node --env-file=.env scripts/<name>.mjs` and are the only code outside edge functions that touches the service-role key.

Security Posture

25

Defence in layers: the database enforces tenancy and roles on every query, the clients add hardening on top, and everything sensitive leaves a trace.

Database floor

RLS on every table and on storage paths (`<org_slug>/...`). Privilege-escalation triggers on profiles. Soft deletes. Capability checks are UI-only sugar above this floor — never the security boundary.

Credentials

Web: passwords + optional TOTP 2FA, HttpOnly cookie sessions. Mobile: bcrypt-hashed 4-digit PINs, 5-fails/15-min lockout, timing-normalised responses, every attempt recorded. API tokens hashed at rest, plaintext shown once.

Web hardening

Strict CSP (only Supabase + map tiles), HSTS 2 years with subdomains, X-Frame-Options DENY, COOP same-origin, restrictive Permissions-Policy. CSRF neutralised by HttpOnly JWT cookies.

Traceability

Audit log on sign-ins, PIN attempts, user/role/org changes, capability edits and manager decisions — actor, role, target, IP, timestamp. Webhook deliveries recorded. Sentry error monitoring opt-in.

Key custody

The service-role key exists only inside edge functions and ops scripts — never in a browser or phone. Apps use the anon key and live entirely under RLS.

Known trade-offs

Auth emails are unique platform-wide (Supabase constraint) — accepted; mobile identity is org + employee number anyway. NFC needs the dev build, not Expo Go.

WARNING — THE TWO REAL KEYS TO GUARD

A leaked **service-role key** bypasses RLS entirely, and a leaked **org admin password** owns that tenant. Rotate the first on schedule; protect the second with 2FA. Everything else has a blast radius designed to be small.

Appendix

26

Demo accounts, a glossary of platform terms, and the core data model at a glance.

A.1 Demo accounts (rotate before production)

SURFACE	IDENTITY	CREDENTIAL	ROLE
Web	super@digilog360.com	Super123!	super_user
Web	admin@digilog360.com	Admin123!	admin
Web	manager@digilog360.com	Manager123!	manager
Web	control@digilog360.com	Control123!	control_room
Web	supervisor@digilog360.com	Supervisor123!	supervisor
Mobile	org digilog-demo · emp GRD001 / GRD002	PIN (seeded)	guard
Mobile	org digilog-demo · emp SUP001	PIN (seeded)	supervisor

A.2 Glossary

TERM	MEANING
Occurrence / OB	A logged incident; OB is its sequential Occurrence Book number (e.g. OB-2026-04181)
Organisation (org)	A tenant — one client company with fully isolated data
Site	A physical guarded location within an org
SLA	Service-level agreement: the resolve-by and update-every clocks per severity
Breach / Update due	Past resolve deadline / past update interval — red and amber on the boards
Capability	A feature key (e.g. <code>patrols.scan</code>) granted per role per org in the permissions grid
RLS	Row-level security — the database rules that hard-enforce org and role isolation
Checkpoint	A physical scan point with a QR label, NFC tag and/or GPS geofence
Acknowledgement	A manager's recorded decision on an occurrence: acknowledged, escalated or rejected
Handover	A signed shift-change summary the incoming guard must acknowledge
Edge function	A serverless backend function holding the privileged key for admin operations
Offline queue	On-device store of occurrences logged without signal, auto-synced later
Email outbox	Durable queue of alert emails, drained within a minute of any event
Delivery ledger	Immutable per-recipient record of every email sent, with its Resend message ID

A.3 Core data model

CLUSTER	TABLES
Tenancy & people	<code>organizations</code> · <code>sites</code> · <code>profiles</code> · <code>capabilities</code> · <code>role_capabilities</code>
Occurrences	<code>occurrences</code> · <code>occurrence_updates</code> · <code>occurrence_reports</code> · <code>occurrence_images</code> · <code>occurrence_comments</code> · <code>manager_acknowledgements</code>
Patrols	<code>patrol_routes</code> · <code>checkpoints</code> · <code>route_checkpoints</code> · <code>patrols</code> · <code>checkpoint_scans</code> · <code>patrol_schedules</code> · <code>expected_patrols</code> · <code>guard_positions</code>
Field ops	<code>shifts</code> · <code>shift_handovers</code> · <code>visitors</code> · <code>keys</code> · <code>key_handovers</code> · <code>tasks</code> · <code>task_updates</code>
Platform	<code>notifications</code> · <code>saved_views</code> · <code>audit_log</code> · <code>pin_attempts</code> · <code>org_sla_overrides</code> · <code>org_occurrence_types</code> · <code>org_webhooks</code> · <code>webhook_deliveries</code> · <code>api_tokens</code> · <code>org_email_settings</code> · <code>email_outbox</code> · <code>email_log</code>